Explicar

Estado	Finalizado
Comenzado	lunes, 2 de noviembre de 2026, 19:44
Completado	lunes, 2 de noviembre de 2026, 21:43
Duración	1 hora 59 minutos

Información

Lectura inicial

Sección 00 — Presentación

Este formulario documenta la propuesta de desarrollo del Trabajo Práctico Integrador 2026. Sus respuestas se utilizarán para acordar el alcance particular del equipo y preparar los criterios específicos y las guías de seguimiento y corrección. No constituye una evaluación de conocimientos ni tiene calificación que afecte la nota del TPI. Su presentación completa es condición necesaria para habilitar la corrección. Las modificaciones posteriores del alcance deberán acordarse con la cátedra. Respondan con precisión y mantengan consistencia entre procesos, servicios, recursos y datos.

Pregunta **1**

Finalizado

Sin calificar

Equipo y responsables

Sección 01 — Identificación

Indiquen nombre del proyecto, identificación del equipo, comisión, integrantes (nombre y apellido) y responsable de contacto. Señalen el repositorio principal del trabajo, si ya está disponible.

CampusReservas
Grupo 159
Cipolla Delcantare Luciano - 408808 3k1
Fiorani Facundo - 408832 3k1
Gamond Santiago – 408814 3k8
Purro Ezequiel - 409476 3k1
Responsable de contacto (Todos)

Pregunta **2**

Finalizado

Sin calificar

Resumen del proyecto

Sección 02 — Descripción breve del proyecto

Describan en un máximo orientativo de 200 a 300 palabras qué solución desarrollarán, a quién está destinada, cuál es su principal aporte y qué resultado concreto permitirá obtener.

CampusReservas permitirá a una facultad gestionar las reservas de sus espacios: aulas, laboratorios, salas y canchas al aire libre. Está destinado a estudiantes y docentes, que podrán consultar los espacios, buscar cuáles están disponibles y gestionar sus propias reservas, y a bedelía, que administrará el catálogo y aprobará o rechazará los eventos.

Reservar un espacio no es solo anotar un nombre en una agenda: cada pedido debe respetar el horario de apertura, la capacidad, las actividades admitidas, el equipamiento y las demás reservas, además de las reglas propias de cada tipo de espacio. La solución validará todas esas condiciones de forma automática y, en el caso de las canchas, consultará el pronóstico de lluvia en Open-Meteo, un dato externo que cambia hora a hora.

Su principal aporte es que no se limita a registrar reservas: combina datos propios (catálogo, equipamiento y ocupación) con datos externos para decidir si una reserva es válida y sugerir los espacios que realmente sirven para lo que el usuario necesita. Como ningún servicio puede resolver esto por sí solo, el sistema se organizará en dos microservicios colaborativos, ms-espacios y ms-reservas, detrás de un API Gateway que concentrará la seguridad, con autenticación y roles gestionados por Keycloak.

Como resultado concreto, se obtendrá un backend desplegable con Docker Compose y documentado con Swagger, que permitirá crear reservas validadas, evitar superposiciones incluso ante pedidos simultáneos, gestionar la aprobación de eventos y obtener sugerencias de espacios disponibles.

Pregunta **3**

Finalizado

Sin calificar

Problema que se busca resolver

Sección 03 — Problema y contexto

Describan el problema actual, sus causas o dificultades principales, a quiénes afecta, qué actores intervienen y por qué resulta pertinente resolverlo mediante el sistema propuesto. Diferencien el problema de la solución técnica.

Problema: En una facultad, los espacios son un recurso compartido por muchas personas y actividades, y reservarlos no es solo anotar un nombre en una agenda. Para saber si un espacio sirve, hay que comprobar a la vez si está abierto en ese horario, si tiene lugar para todos los asistentes, si admite la actividad, si cuenta con el equipamiento necesario y si no está ocupado. Cuando estas comprobaciones no se hacen de forma unificada, pueden aceptarse reservas que no se podrán cumplir, producirse superposiciones o perderse tiempo buscando un lugar adecuado.

Causas y dificultades principales:

- La información necesaria es de distinta naturaleza: las características de cada espacio, su ocupación y, en las canchas al aire libre, el clima.
- Las condiciones cambian según el tipo de espacio. Por ejemplo, un laboratorio requiere anticipación para preparar los equipos, y una cancha depende del pronóstico de lluvia.
- Parte de esa información cambia constantemente: la ocupación varía con cada pedido y el pronóstico, hora a hora.
- Dos personas pueden pedir el mismo espacio al mismo tiempo.
- No todas las reservas se resuelven igual: los eventos necesitan una autorización previa.

A quiénes afecta y qué actores intervienen: Afecta a estudiantes y docentes, que necesitan encontrar un espacio adecuado y saber si su reserva es válida, y a bedelía, que debe administrar los espacios y decidir sobre los eventos. El servicio meteorológico interviene como fuente externa del pronóstico.

Pertinencia. Resolverlo mediante un sistema es pertinente porque son muchas reglas, que deben aplicarse siempre de la misma manera y con datos actualizados en el momento de cada pedido. Un sistema también puede cruzar esas condiciones para indicar qué espacios sirven, en lugar de que cada persona deba averiguarlo.

Problema y solución técnica. El problema es organizacional: decidir de forma confiable quién puede usar qué espacio, cuándo y para qué, y existiría con cualquier tecnología. La solución técnica es el medio elegido para resolverlo: un backend de microservicios con API Gateway, autenticación y consulta a una API meteorológica.

Pregunta **4**

Finalizado

Sin calificar

Alcance funcional, reglas y proceso de negocio

Sección 04 — Alcance funcional y proceso principal

Describan el alcance funcional comprometido del MVP y el proceso de negocio principal en una respuesta integrada.

1. Funcionalidades: enumeren las funcionalidades incluidas, indicando identificador, nombre, actor, comportamiento esperado, resultado observable y prioridad. Distingan qué queda fuera del alcance.

2. Reglas de negocio: para cada regla indiquen identificador, condición o disparador, comportamiento esperado, posibles rechazos y funcionalidad asociada. Incluyan estados y transiciones cuando corresponda.

3. Proceso principal: expliquen de extremo a extremo el evento inicial, participantes, pasos, decisiones, excepciones relevantes y resultado final. Incorporen un diagrama con la notación que prefieran (BPMN, actividades, secuencia, estados u otra), su definición textual editable, su representación gráfica y una breve explicación. Pueden utilizar PlantUML, Mermaid o una herramienta equivalente.

Relacionen explícitamente las funcionalidades y reglas con los pasos del proceso.

1. Funcionalidades

- F1 - Administrar espacios: Ejecutado por ADMIN. Su comportamiento es el alta, modificación y baja. Da como resultado 201 o 200 (catálogo actualizado). Prioridad: Alta.
- F2 - Consultar espacios: Ejecutado por USUARIO y ADMIN. Permite ver lista o detalle. Da como resultado 200, o 404 si no existe. Prioridad: Media.
- F3 - Buscar disponibles: Ejecutado por USUARIO. Cruza espacios compatibles con la ocupación. Da como resultado 200 con los espacios libres que sirven. Prioridad: Alta.
- F4 - Crear reserva: Ejecutado por USUARIO. Valida y guarda a nombre del token. Da como resultado 201 CONFIRMADA o PENDIENTE. Prioridad: Alta.
- F5 - Aprobar o rechazar evento: Ejecutado por ADMIN. Resuelve una reserva PENDIENTE. Da como resultado 200 CONFIRMADA o RECHAZADA. Prioridad: Alta.
- F6 - Consultar reservas propias: Ejecutado por USUARIO. Lista las reservas asociadas al token. Da como resultado 200 con la lista. Prioridad: Media.
- F7 - Cancelar reserva propia: Ejecutado por USUARIO dueño. Cancela la reserva antes del inicio. Da como resultado 200 CANCELADA, y libera el espacio. Prioridad: Media.
- F8 - Consultar todas las reservas: Ejecutado por ADMIN. Lista las reservas con filtro por estado. Da como resultado 200 con la lista. Prioridad: Alta.

- F9 - Autenticación y rol: Ejecutado por USUARIO y ADMIN. Keycloak emite el JWT y el gateway lo valida. Como resultado, reenvía el pedido o devuelve 401 o 403. Prioridad: Alta.

Fuera del alcance: frontend, pagos, notificaciones, gestión de usuarios, modificar reservas, reservas recurrentes, reservas hechas por bedelía, registro de uso, vencimiento automático y revalidación de reservas.

2. Reglas de negocio

- RN1: (Crear o buscar) Debe tener inicio futuro, a 14 días como máximo. Rechazo: 422. (F3, F4)
- RN2: (Crear o buscar) La duración debe ser de entre 30 minutos y 4 horas, en el mismo día. Rechazo: 422. (F3, F4)
- RN3: (Validar espacio) Debe estar dentro del horario de apertura. Rechazo: 422 (no se sugiere). (F3, F4)
- RN4: (Validar cancha) Lluvia pronosticada de hasta 60 % en cada hora. Rechazo: 422 o 503 (no se sugiere). (F3, F4)
- RN5: (Crear o buscar) Cantidad de asistentes debe estar dentro de la capacidad. Rechazo: 422 (no se sugiere). (F3, F4)
- RN6: (Crear o buscar) No puede haber superposición con una reserva PENDIENTE o CONFIRMADA. Rechazo: 409 (se descarta). (F3, F4)
- RN7: (Cancelar) Solo lo puede hacer el dueño y antes del horario de inicio. Rechazo: 403 o 422. (F7)
- RN8: (Pedido con equipamiento) El espacio debe tener todo lo solicitado. Rechazo: 422 (no se sugiere). (F3, F4)
- RN9: (Validar actividad) El espacio debe admitir la actividad solicitada. Rechazo: 422 (no se sugiere). (F3, F4)
- RN10: (Validar laboratorio) Solo de lunes a viernes y con 24 horas de anticipación mínima. Rechazo: 422 (no se sugiere). (F3, F4)
- RN11: (Validar sala) Tiempo máximo de 2 horas. Rechazo: 422 (no se sugiere). (F3, F4)
- RN12: (Validar cancha) Los turnos deben ser de horas enteras. Rechazo: 422 (no se sugiere). (F3, F4)
- RN13: (Reserva válida) Las reservas de tipo EVENTO nacen como PENDIENTE, las demás como CONFIRMADA. No genera rechazo. (F4, F5)
- RN14: (Cambio de estado) Solo se permiten las transiciones marcadas en el diagrama. Rechazo: 409 o 422. (F5, F7)
- RN15: (Pedidos simultáneos) Ante concurrencia gana un solo pedido, gestionado por control de versión. Rechazo: 409. (F4, F5, F7)

Estados y transiciones:

```
@startuml estados-reserva
!pragma layout smetana
```

```
skinparam shadowing false
hide empty description

title CampusReservas — Estados de una reserva

[*] --> PENDIENTE : se crea con actividad EVENTO
[*] --> CONFIRMADA : se crea con otra actividad
PENDIENTE --> CONFIRMADA : ADMIN aprueba\n(antes del inicio)
PENDIENTE --> RECHAZADA : ADMIN rechaza
PENDIENTE --> CANCELADA : dueño cancela\n(antes del inicio)
CONFIRMADA --> CANCELADA : dueño cancela\n(antes del inicio)
RECHAZADA --> [*]
CANCELADA --> [*]

@enduml
```

3. Proceso principal: reservar un espacio

Evento inicial: un estudiante o docente necesita un espacio. Precondición: bedelía cargó el catálogo (F1).

Participantes: Usuario, Bedelía, Keycloak, API Gateway, ms-reservas, ms-espacios y Open-Meteo.

Resultado final: reserva CONFIRMADA, o fin sin ocupar el espacio (rechazo, RECHAZADA o CANCELADA).

Decisiones: token y rol válidos (P2); sabe qué espacio quiere (P3); alguna opción le sirve (P6); pasó las validaciones (P8 a P10); otro pedido ganó la agenda (P11); es un EVENTO (P11); bedelía aprueba (P13); el dueño cancela (P15).

- P1: El Usuario obtiene un JWT de Keycloak. (F9)
- P2: El gateway valida token y rol, y agrega el header X-Usuario. (F9)
- P3a: Si ya sabe qué espacio quiere, toma el ID del catálogo. (F2)
- P3b: Si no sabe el espacio, indica la franja horaria, actividad, asistentes y equipamiento necesario. (F3)
- P4: ms-reservas controla la franja horaria. (F3 | Reglas: RN1, RN2)
- P5: ms-espacios filtra aquellos que sean compatibles. (F3 | Reglas: RN3, RN4, RN5, RN8 a RN12)
- P6: ms-reservas descarta los espacios que están ocupados. (F3 | Reglas: RN6)
- P7: El Usuario pide crear la reserva definitivamente. (F4)
- P8: ms-reservas toma la agenda del espacio y controla la franja horaria. (F4 | Reglas: RN15, RN1, RN2)
- P9: ms-espacios valida las condiciones específicas del espacio solicitado. (F4 | Reglas: RN3, RN4, RN8 a RN12)

- P10: ms-reservas controla la capacidad y verifica que no haya superposición. (F4 | Reglas: RN5, RN6)
- P11: ms-reservas define el estado de la reserva y guarda el registro. (F4 | Reglas: RN13, RN15)
- P12: Bedelía lista las reservas que están pendientes. (F8)
- P13: Bedelía toma una decisión (aprueba o rechaza). (F5)
- P14: La reserva cambia de estado a CONFIRMADA o RECHAZADA. (F5 | Reglas: RN14, RN15)
- P15: El Usuario consulta el estado de sus reservas. (F6)
- P16: Si el dueño lo solicita, la reserva pasa a estado CANCELADA. (F7 | Reglas: RN7, RN14, RN15)

Excepciones:

- Token o rol inválido: 401 o 403.
- Datos mal formados: 400.
- Espacio inexistente: 404.
- Regla incumplida: 422.
- Franja ocupada o pedidos simultáneos: 409.
- ms-espacios caído: 503 al consultar el catálogo, buscar o crear.
- Open-Meteo caído: 503 al reservar una cancha; la búsqueda la omite.

Diagrama: actividades con un carril por participante; muestra quién hace cada paso y dónde colaboran los microservicios.

```
@startuml proceso-reserva
skinparam shadowing false
skinparam ActivityBackgroundColor #F5F7FA
skinparam ActivityBorderColor #4A5A6A
skinparam ActivityDiamondBackgroundColor #FFF4D6
skinparam ArrowColor #4A5A6A
skinparam SwimlaneTitleFontStyle bold

title CampusReservas — Proceso principal: reservar un espacio

|Usuario|
start
note left
  Precondición: bedelía cargó
  el catálogo de espacios (F1)
end note
:P1. Inicia sesión en Keycloak
y obtiene un JWT;

|API Gateway|
:P2. Valida el JWT y el rol, y agrega
X-Usuario (en cada pedido);
if (¿Token y rol válidos?) then (no)
  #FFE3E0:401 o 403;
```

```
  stop
else (sí)
endif

|Usuario|
if (¿Ya sabe qué espacio quiere?) then (sí)
  :P3a. Consulta el catálogo
  y toma el id del espacio;
else (no)
  repeat
    |Usuario|
    :P3b. Indica franja, actividad,
    asistentes y equipamiento;
    |ms-reservas|
    :P4. Controla RN1 y RN2;
    |ms-espacios|
    :P5. Filtra los compatibles
    (RN3, RN4, RN5, RN8 a RN12);
    |ms-reservas|
    :P6. Descarta los ocupados (RN6)
    y devuelve las sugerencias;
    |Usuario|
  repeat while (¿Alguna opción le sirve?) is (no) not (sí)
endif

|Usuario|
:P7. Pide crear la reserva;

|ms-reservas|
:P8. Toma la agenda del espacio (RN15)
y controla RN1 y RN2;

|ms-espacios|
:P9. Valida el espacio: RN3, RN9, RN8
y las reglas de su tipo (RN10, RN11, RN12);
note right
  Solo canchas: consulta
  Open-Meteo (RN4)
end note

|ms-reservas|
:P10. Controla capacidad (RN5)
y superposición (RN6);
if (¿Pasó todas las validaciones?) then (no)
  #FFE3E0:404, 409, 422 o 503
  No guarda nada;
  stop
else (sí)
endif

:P11. Define el estado inicial (RN13)
y guarda en una transacción (RN15);
if (¿Otro pedido ganó la agenda?) then (sí)
```

```
  #FFE3E0:409, puede reintentar
  No guarda nada;
  stop
else (no)
endif

if (Estado inicial) then (PENDIENTE: es un EVENTO)
  |Bedelía|
  :P12. Lista las reservas pendientes;
  :P13. Revisa el evento y decide;
  |ms-reservas|
  if (Decisión de bedelía) then (rechaza)
    #E8E8E8:P14. Pasa a RECHAZADA (RN14)
    y libera el espacio;
    stop
  else (aprueba)
    :P14. Pasa a CONFIRMADA (RN14);
  endif
else (CONFIRMADA)
endif

|Usuario|
:P15. Consulta sus reservas;
if (¿Cancela antes del inicio?) then (sí)
  |ms-reservas|
  #E8E8E8:P16. Controla RN7 y RN14, pasa a
  CANCELADA y libera el espacio;
  stop
else (no)
  |Usuario|
  #DFF3E3:Reserva CONFIRMADA:
  espacio asegurado para la actividad;
  stop
endif

@enduml
```

Pregunta **5**

Finalizado

Sin calificar

Diagrama C4 — Nivel 1 (Contexto)

Sección 05 — Arquitectura: Contexto

Presenten el diagrama de Contexto del modelo C4, identificando el sistema que construirán, los actores y los sistemas externos con los que interactúa. Incluyan la definición textual editable del diagrama, su representación gráfica y una descripción de cada elemento y relación. Se recomienda PlantUML con C4-PlantUML, Mermaid u otra herramienta equivalente.

Elementos:

- Usuario: estudiante o docente (rol USUARIO). Busca, reserva y cancela.
- Administrador: bedelía (rol ADMIN). Administra espacios y resuelve eventos.
- CampusReservas: backend REST de espacios y reservas.
- Keycloak: Identity Provider externo; autentica y emite JWT.
- Open-Meteo: API pública de pronóstico por hora.

Relaciones:

- Usuario y Administrador → Keycloak (OIDC): obtienen un JWT.
- Usuario y Administrador → CampusReservas (HTTP/JSON + JWT): operan el sistema.
- CampusReservas → Keycloak (HTTP/JWKS): obtiene las claves públicas.
- CampusReservas → Open-Meteo (HTTPS/JSON): consulta la lluvia, solo para canchas.

```
@startuml c1-contexto
!pragma layout smetana
!include <C4/C4_Context>

title CampusReservas — Diagrama de Contexto (C1)

Person(usuario, "Usuario", "Estudiante o docente. Rol USUARIO. Consulta espacios, busca
Person(admin, "Administrador", "Bedelía. Rol ADMIN. Administra los espacios, aprueba o

System(campus, "CampusReservas", "Backend de reservas de espacios de la facultad. Valid

System_Ext(keycloak, "Keycloak", "Identity Provider. Guarda usuarios y roles, autentica
System_Ext(openmeteo, "Open-Meteo", "API pública de pronóstico del tiempo por coordenad

Rel_D(usuario, campus, "Consulta espacios, busca\ndisponibles, reserva y cancela", "HTT
Rel_U(admin, campus, "Administra espacios, aprueba\no rechaza eventos y lista reservas"
Rel(usuario, keycloak, "Inicia sesión\ny obtiene un JWT", "OIDC")
Rel(admin, keycloak, "Inicia sesión\ny obtiene un JWT", "OIDC")
Rel_R(campus, keycloak, "Obtiene las claves públicas\npara validar los JWT", "HTTP/JWKS
Rel_L(campus, openmeteo, "Consulta la probabilidad de lluvia\npor hora (solo canchas)",
```

```
SHOW_LEGEND()
@enduml
```

diagrama de Contexto del modelo C4.png

Pregunta **6**

Finalizado

Sin calificar

Diagrama C4 — Nivel 2 (Contenedores)

Sección 06 — Arquitectura: Contenedores

Presenten el diagrama de Contenedores del modelo C4. Identifiquen microservicios, aplicaciones, bases de datos y demás unidades ejecutables o almacenes relevantes; indiquen responsabilidades, tecnologías principales y comunicaciones. Incluyan definición textual editable, representación gráfica y descripción de componentes y relaciones. Mantengan los nombres consistentes con el resto de las respuestas.

- API Gateway (8080, publicado)
  - *Tecnología:* Spring Cloud Gateway 5, Java 25
  - *Responsabilidad:* Entrada única: ruteo, JWT, rol y X-Usuario.
- ms-reservas (8082, interno)
  - *Tecnología:* Spring Boot 4.1, JPA
  - *Responsabilidad:* Reservas, estados, ocupación y concurrencia.
- ms-espacios (8081, interno)
  - *Tecnología:* Spring Boot 4.1, JPA
  - *Responsabilidad:* Catálogo, reglas por tipo y clima.
- reservas-db
  - *Tecnología:* H2 en archivo
  - *Responsabilidad:* Reservas y agenda.
- espacios-db
  - *Tecnología:* H2 en archivo
  - *Responsabilidad:* Espacios, equipamiento y actividades.
- campus-commons
  - *Tecnología:* JAR Maven
  - *Responsabilidad:* ApiResponse, excepciones y errores comunes.
- Keycloak 26 (8180, publicado)
  - *Tecnología:* OIDC y JWT
  - *Responsabilidad:* Identity Provider externo.
- Open-Meteo
  - *Tecnología:* HTTPS/JSON
  - *Responsabilidad:* Pronóstico externo.

Comunicaciones:
Clientes → Keycloak (OIDC) y → API Gateway (HTTP/JSON + Bearer).
API Gateway → Keycloak (JWKS) y → microservicios (HTTP/JSON + X-Usuario).
ms-reservas → ms-espacios (HTTP síncrono con RestClient), en una sola dirección.
ms-espacios → Open-Meteo (HTTPS/JSON), solo para canchas.
Cada microservicio → su base (JPA/JDBC) y → campus-commons (dependencia Maven).
@startuml c2-contenedores
!pragma layout smetana
!include <C4/C4_Container>
title CampusReservas — Diagrama de Contenedores (C2)
AddElementTag("libreria", $bgColor="#7A8FA6", $fontColor="#FFFFFF", $legendText="librería Java (no se despliega)")
Person(usuario, "Usuario", "Estudiante o docente. Busca espacios, reserva y cancela sus reservas.")
Person(admin, "Administrador", "Bedelía. Administra espacios, aprueba o rechaza eventos y consulta todas las reservas.")
System_Ext(openmeteo, "Open-Meteo", "API pública de pronóstico del tiempo por coordenadas.")
Elemento Tecnología Responsabilidad
ms-reservas (8082, interno)
Spring Boot 4.1, JPA Reservas, estados, ocupación y concurrencia.
ms-espacios (8081, interno)
Spring Boot 4.1, JPA Catálogo, reglas por tipo y clima.
reservas-db H2 en archivo Reservas y agenda.
espacios-db H2 en archivo Espacios, equipamiento y actividades.
campus-commons JAR Maven ApiResponse, excepciones y errores comunes.
Keycloak 26 (8180, publicado)
OIDC y JWT Identity Provider externo.
Open-Meteo HTTPS/JSON Pronóstico externo.
Versión corta
Page 13 of 23
Boundary(docker, "Despliegue Docker Compose — red interna (trusted zone)") {
Container_Ext(keycloak, "Keycloak 26", "Identity Provider", "Puerto publicado 8180. Realm campus: usuarios y roles USUARIO y ADMIN. Emite JWT (OIDC).")
System_Boundary(campus, "CampusReservas") {
Container(gateway, "API Gateway", "Spring Cloud Gateway 5, Spring

```
Boot 4.1, Java 25", "Único punto de entrada (puerto publicado 8080). Ruteo
por path. Security Filter: valida el JWT y el rol por ruta; agrega el header
X-Usuario.")
Container(msreservas, "ms-reservas", "Spring Boot 4.1, Java 25,
Spring Data JPA", "Puerto interno 8082. Ciclo de vida de la reserva: crear,
aprobar, rechazar, cancelar y listar. Busca espacios disponibles. Cadena de
validaciones, estados y concurrencia.")
Container(msespacios, "ms-espacios", "Spring Boot 4.1, Java 25,
Spring Data JPA", "Puerto interno 8081. ABM de espacios. Valida una franja
con una estrategia por tipo de espacio. Lista los espacios compatibles.
Consulta el pronóstico.")
ContainerDb(reservasdb, "reservas-db", "H2 embebida en archivo",
"Reservas y agenda por espacio (versión para concurrencia). Solo la usa msreservas.")
ContainerDb(espaciosdb, "espacios-db", "H2 embebida en archivo",
"Espacios, equipamiento y actividades admitidas. Solo la usa ms-espacios.")
Container(commons, "campus-commons", "JAR Maven instalado en .m2",
"ApiResponse, excepciones de negocio y manejo de errores común.",
$tags="libreria")
}
}
Rel(usuario, keycloak, "Inicia sesión y obtiene el JWT", "OIDC/HTTP")
Rel(admin, keycloak, "Inicia sesión y obtiene el JWT", "OIDC/HTTP")
Rel(usuario, gateway, "Consulta espacios y opera sus reservas", "HTTP/JSON +
Bearer JWT")
Rel(admin, gateway, "Administra espacios y aprueba eventos", "HTTP/JSON +
Bearer JWT")
Rel_L(gateway, keycloak, "Obtiene las claves públicas", "JWKS/HTTP")
Rel(gateway, msreservas, "Rutea /api/reservas/**", "HTTP/JSON + X-Usuario")
Rel(gateway, msespacios, "Rutea /api/espacios/**", "HTTP/JSON + X-Usuario")
Rel_R(msreservas, msespacios, "Valida un espacio y pide los compatibles",
"HTTP/JSON, RestClient")
Rel(msreservas, reservasdb, "Lee y escribe", "JPA/JDBC")
Rel(msespacios, espaciosdb, "Lee y escribe", "JPA/JDBC")
Rel(msespacios, openmeteo, "Pide el pronóstico de lluvia (solo canchas)",
"HTTPS/JSON")
Versión corta
Page 14 of 23
Rel(msreservas, commons, "Dependencia Maven")
Rel(msespacios, commons, "Dependencia Maven")
SHOW_LEGEND()
@enduml
```

diagrama de Contenedores del modelo C4.png

Pregunta **7**

Finalizado

Sin calificar

Recursos y endpoints por microservicio

Sección 07 — Arquitectura: APIs

Completen una tabla por cada microservicio con todos los endpoints comprometidos. Utilicen las columnas: Microservicio | Recurso | Método HTTP | Endpoint | Operación de negocio | Seguridad. La operación debe describir el efecto funcional, no repetir el verbo HTTP. Ejemplo:

```
| Microservicio | Recurso | Método HTTP | Endpoint | Operación de negocio | Seguridad |
|---|---|---|---|---|---|
| Reservas | Reservas | POST | /reservas | Registrar una reserva | Usuario autenticado
| Reservas | Reservas | GET | /reservas/{id} | Consultar una reserva | Usuario autoriza
| Reservas | Reservas | PATCH | /reservas/{id}/confirmacion | Confirmar una reserva | O
```

Incluyan endpoints de consulta, modificación y acciones específicas de negocio cuando correspondan.

ms-espacios

Microservicio	Recurso	Método HTTP	Endpoint	Operación de negocio	Seguridad
ms-espacios	Espacios	GET	/api/espacios	Consultar el catálogo de espacios	USUARIO o ADMIN
ms-espacios	Espacios	GET	/api/espacios/{id}	Consultar el detalle de un espacio	USUARIO o ADMIN
ms-espacios	Espacios	POST	/api/espacios	Dar de alta un espacio en el catálogo	ADMIN
ms-espacios	Espacios	PUT	/api/espacios/{id}	Actualizar los datos de un espacio	ADMIN
ms-espacios	Espacios	DELETE	/api/espacios/{id}	Dar de baja un espacio del catálogo	ADMIN
ms-espacios	Validación de franja	GET	/api/espacios/{id}/validacion	Verificar si un espacio se puede usar en una franja: horario, actividad, equipamiento, reglas de su tipo y clima, e informar el motivo si no sirve	USUARIO o ADMIN por el gateway; ms-reservas por la red interna, al crear una reserva
ms-espacios	Espacios compatibles	GET	/api/espacios/compatibles	Obtener los espacios que cumplen capacidad, actividad, equipamiento, horario y clima para una franja	USUARIO o ADMIN por el gateway; ms-reservas por la red interna, al buscar disponibles

ms-reservas

Microservicio	Recurso	Método HTTP	Endpoint	Operación de negocio	Seguridad
ms-reservas	Reservas	POST	/api/reservas	Registrar una reserva a nombre del usuario del token: queda CONFIRMADA, o PENDIENTE si es un evento	USUARIO
ms-reservas	Reservas	GET	/api/reservas/mias	Consultar las reservas propias	USUARIO

(solo las propias, por X-Usuario) |
ms-reservas	Reservas	GET	/api/reservas	Consultar todas las reservas, con filtro opcional por estado	ADMIN
ms-reservas	Reservas	PATCH	/api/reservas/{id}/aprobar	Aprobar un evento pendiente antes de su inicio, que pasa a CONFIRMADA	ADMIN
ms-reservas	Reservas	PATCH	/api/reservas/{id}/rechazar	Rechazar un evento pendiente y liberar el espacio	ADMIN
ms-reservas	Reservas	PATCH	/api/reservas/{id}/cancelar	Cancelar una reserva propia antes de su inicio y liberar el espacio	USUARIO (solo el dueño, RN7)
ms-reservas	Disponibilidad	GET	/api/reservas/disponibilidad	Sugerir los espacios compatibles con el pedido que están libres en la franja	USUARIO

Pregunta **8**

Finalizado

Sin calificar

DER y decisiones de persistencia

Sección 08 — Arquitectura: Persistencia

Presenten el Diagrama Entidad-Relación de las bases de datos comprometidas, con entidades, atributos clave, relaciones y cardinalidades. Acompañen el diagrama con una definición textual editable y una breve descripción. Expliquen qué microservicio administra cada conjunto de datos y las principales decisiones de persistencia. Si existen varias bases, documenten cada una.

Hay dos bases H2 en archivo, sin claves foráneas entre sí. Las líneas continuas son FK reales y las punteadas, relaciones lógicas.

```
@startuml der-persistencia
!pragma layout smetana
hide circle
hide methods
skinparam linetype ortho
skinparam shadowing false
skinparam roundCorner 6
skinparam defaultFontName Arial
skinparam packageStyle rectangle
skinparam ArrowColor #333333
skinparam class {
    BackgroundColor #FFFFFF
    BorderColor #333333
    HeaderBackgroundColor #E8EEF5
}

title CampusReservas — Diagrama Entidad-Relación (una base por microservicio)

package "espacios-db (H2) — dueño: ms-espacios" as espaciosdb {
    entity "espacios" as espacios {
        * id : BIGINT <<PK, identity>>
        --
        nombre : VARCHAR(255)
        tipo : VARCHAR(255) <<CHECK>>
        * capacidad : INTEGER
        * latitud : DOUBLE
        * longitud : DOUBLE
        hora_apertura : TIME
        hora_cierre : TIME
    }

    entity "espacio_equipamiento" as equipamiento {
        * espacio_id : BIGINT <<FK, UK>>
```

```
        {field} equipamiento : VARCHAR(255) <<UK, CHECK>>
    }

    entity "espacio_actividades" as actividades {
        * espacio_id : BIGINT <<FK, UK>>
        {field} actividad : VARCHAR(255) <<UK, CHECK>>
    }
}

package "reservas-db (H2) — dueño: ms-reservas" as reservasdb {
    entity "reservas" as reservas {
        * id : BIGINT <<PK, identity>>
        --
        espacio_id : BIGINT <<ref. lógica>>
        espacio_nombre : VARCHAR(255) <<copia>>
        usuario : VARCHAR(255)
        inicio : TIMESTAMP
        fin : TIMESTAMP
        * asistentes : INTEGER
        actividad : VARCHAR(255) <<CHECK>>
        estado : VARCHAR(255) <<CHECK>>
        * version : BIGINT <<@Version>>
    }

    entity "agenda_espacio" as agenda {
        * espacio_id : BIGINT <<PK, ref. lógica>>
        --
        * version : BIGINT <<@Version>>
    }
}

espacios ||--o{ equipamiento : "tiene"
espacios ||--|{ actividades : "admite"

agenda ||..|{ reservas : "ordena (mismo espacio_id, sin FK)"
espacios ||..o{ reservas : "reservado en (por API, sin FK)"
espacios ||..o| agenda : "se agenda en (por API, sin FK)"

legend right
    Línea continua: clave foránea real, dentro de la misma base.
    Línea punteada: relación lógica, sin FK; no la controla la base.
    * = NOT NULL en la base. UK = UNIQUE compuesta (espacio_id, valor).
    CHECK = columna limitada a los valores del enum al crear la tabla.
endlegend
@enduml
```

Administración de datos:

- ms-espacios → espacios-db: catálogo, equipamiento y actividades.
- ms-reservas → reservas-db: reservas y agenda por espacio.

Principales decisiones:

- Una base por microservicio: cada servicio es dueño de sus datos.

- H2 en archivo: no suma contenedores y persiste entre reinicios.
- espacio_id sin FK y copia del nombre: listar y cancelar funcionan con ms-espacios caído.
- Enums y @ElementCollection con Set: listas cortas de valores sin identidad propia.
- agenda_espacio con @Version: las reservas simultáneas chocan en una fila común y una recibe 409.
- @Version en reservas: aprobar y cancelar a la vez no se pisan.
- Sin tabla de usuarios: la identidad es de Keycloak.

Pregunta **9**

Finalizado

Sin calificar

Librería común y reutilización

Sección 09 — Arquitectura: Librería común

Describan si implementarán una librería común. Indiquen nombre, propósito, componentes que contendrá, servicios que la utilizarán y mecanismo de distribución o dependencia. Justifiquen qué elementos son realmente reutilizables y cuáles deben permanecer dentro de cada microservicio. Si no prevén librería común, explíquenlo y fundamenten la decisión.

Sí, implementamos campus-commons, para que los microservicios respondan y fallen igual.
Componentes: ApiResponse, cinco excepciones de negocio (404, 403, 409, 422 y 503) y GlobalExceptionHandler.
Servicios que la usan: ms-espacios y ms-reservas. El gateway no, porque es reactivo.
Distribución: se instala con mvn install en .m2 y cada servicio la declara en su pom.xml.
Qué se reutiliza: lo que no depende del negocio: formato de respuesta, categorías de error y traducción a HTTP.
Qué queda en cada microservicio: entidades, repositorios, DTOs, TipoActividad, reglas, estrategias, cadena de validaciones, ConcurrenciaExceptionHandler y RestClientConfig.
Compartir dominio acoplaría los servicios.

Pregunta **10**

Finalizado

Sin calificar

Seguridad y autorización

Sección 10 — Implementación y verificación

Documenten el mecanismo de autenticación previsto, roles o permisos, recursos protegidos y comportamiento ante accesos no autorizados. Expliquen cómo podrán comprobarse al menos un acceso permitido y uno rechazado.

Autenticación: OpenID Connect con JWT emitidos por Keycloak. El API Gateway valida firma, vencimiento y emisor, y reemplaza X-Usuario por el usuario del token.

Roles: USUARIO (estudiante o docente) y ADMIN (bedelía). No son acumulativos.

Recursos protegidos:

USUARIO y ADMIN: GET /api/espacios, /api/espacios/{id}, /api/espacios/{id}/validacion y /api/espacios/compatibles

Solo ADMIN: POST, PUT y DELETE de espacios; GET /api/reservas; PATCH aprobar y rechazar

Solo USUARIO: POST /api/reservas; GET /api/reservas/mias y /api/reservas/disponibilidad; PATCH cancelar (solo el dueño)

Cualquier otra ruta se deniega.

Accesos no autorizados: sin token o con token inválido, 401. Con rol sin permiso o ruta no contemplada, 403. Si la reserva es de otro usuario, ms-reservas responde 403. Ningún rechazo modifica datos.

Comprobación con Postman, contra localhost:8080:

- Permitido: GET /api/espacios con token de usuario1 da 200.
- Rechazado sin token: GET /api/espacios da 401.
- Rechazado por rol: POST /api/espacios con token de usuario1 da 403.

Pregunta **11**

Finalizado

Sin calificar

Ejecución y despliegue

Sección 10 — Implementación y verificación

Indiquen tecnologías y versiones principales, dependencias de infraestructura, configuración por entorno y procedimiento de arranque de la solución. Describan qué archivos o instrucciones permitirán ejecutar el backend completo, incluyendo contenedores si corresponde.

Tecnologías: Java 25, Maven 3.9+, Spring Boot 4.1.2, Spring Cloud Gateway 5 (2025.1.x), H2, springdoc-openapi 3.1.1, Keycloak 26.8.1, Docker Compose v2 y campus-commons 1.0.0.
Infraestructura: Keycloak con el realm importado desde keycloak/realm-campus.json, Open-Meteo y la red interna de Docker como trusted zone.
Configuración por entorno: application.yaml con ${VARIABLE:valorPorDefecto}, y variables de entorno en docker-compose.yml. Las principales son OPEN_METEO_URL, CLIMA_UMBRAL, MS_ESPACIOS_URL, MS_RESERVAS_URL y JWK_SET_URI.
Archivos: build.sh, un Dockerfile por aplicación, docker-compose.yml, keycloak/realm-campus.json, pom.xml, application.yaml y README.md.
Arranque:
1. Instalar JDK 25, Maven 3.9+ y Docker con Compose v2.
2. Ejecutar ./build.sh.
3. Ejecutar docker compose up --build.
4. Pedir un token a Keycloak en localhost:8180.
5. Consumir la API en localhost:8080 con Authorization: Bearer <token>.

Pregunta **12**

Finalizado

Sin calificar

Colección de endpoints y escenarios de prueba

Sección 10 — Implementación y verificación

Describan cómo entregarán una colección ejecutable de llamadas a las APIs (por ejemplo, Postman o Bruno). Indiquen cómo organizarán las solicitudes, variables de entorno, autenticación, datos iniciales y orden de ejecución. Identifiquen los escenarios positivos y negativos que permitirán verificar el proceso principal, las reglas de negocio y las integraciones.

Entrega: colección y environment exportados en Docs/, ejecutados contra el gateway.

Variables y autenticación: token_usuario1, token_usuario2 y token_admin1. Cada login contra Keycloak guarda su token con un script en Tests, y los demás requests lo envían como Bearer.

Datos iniciales: usuario1, usuario2 y admin1 desde el realm. Cinco espacios cargados por un CommandLineRunner: Aula 101, Aula 102, Laboratorio 3, Sala de reuniones y Cancha de fútbol.

Orden: login, seguridad, proceso principal, reglas y ciclo de vida, e integraciones.

Escenarios de Prueba:

- Positivo - Buscar clase para 30 con proyector: Resultado esperado 200 con Aula 101.
- Positivo - Reservar Aula 101: Resultado esperado 201 CONFIRMADA.
- Positivo - Reservar EVENTO y aprobarlo: Resultado esperado 201 PENDIENTE; 200 CONFIRMADA.
- Negativo - Sin token o token modificado: Resultado esperado 401.
- Negativo - usuario1 en una ruta de ADMIN: Resultado esperado 403.
- Negativo - Romper RN1 a RN5 y RN8 a RN12: Resultado esperado 422.
- Negativo - Misma franja (RN6) o transición inválida (RN14): Resultado esperado 409.
- Negativo - Cancelar reserva ajena (RN7): Resultado esperado 403.
- Negativo - Dos POST idénticos en paralelo (RN15): Resultado esperado 201 y 409.
- Negativo - Espacio inexistente: Resultado esperado 404.
- Integración - Open-Meteo caído: Resultado esperado 503 en cancha; la búsqueda la omite.
- Integración - ms-espacios apagado: Resultado esperado 503 al crear y buscar; /mias y cancelar funcionan.

Pregunta **13**

Finalizado

Sin calificar

Alcance acordado, desafíos y exclusiones

Sección 11 — Compromiso del equipo

Sinteticen el compromiso concreto del equipo: proceso central que quedará funcionando, funcionalidades obligatorias, integraciones relevantes, desafíos técnicos o funcionales y exclusiones expresas. Identifiquen riesgos o dependencias que puedan afectar el alcance. El compromiso declarado será la base de la validación con la cátedra y de los criterios específicos de corrección.

Proceso central: reservar un espacio y buscar disponibles, con reglas validadas entre ms-reservas y ms-espacios, y aprobación de eventos por bedelía.

Funcionalidades obligatorias: dos microservicios con datos propios, roles USUARIO y ADMIN, RN1 a RN15, cuatro estados de reserva, control de concurrencia, API Gateway con JWT y roles, campus-commons, H2 por servicio, Swagger, patrones Strategy y Chain of Responsibility, Docker Compose y colección de Postman.

Integraciones: Keycloak (identidad), Open-Meteo (lluvia en canchas) y ms-reservas → ms-espacios por REST.

Desafíos: coordinar dos servicios sin ciclos ni base compartida, concurrencia, reglas por tipo de espacio, ciclo de vida de la reserva, tolerancia a fallas y seguridad centralizada.

Exclusiones: frontend, pagos, notificaciones, segunda API externa, estados automáticos, bloqueo temporal, circuit breaker, JWT en microservicios, tokens entre servicios, PostgreSQL y Kubernetes.

Riesgos y dependencias: caída de Open-Meteo o ms-espacios (503 parcial), emisor del token en localhost:8180, versiones recientes del stack, orden de build de campus-commons y avance por fases.

Pregunta **14**

Finalizado

Sin calificar

Criterios de verificación propuestos

Sección 11 — Compromiso del equipo

Describan de 3 a 8 comprobaciones observables que permitan verificar el cumplimiento del compromiso: condición inicial, acción a ejecutar y resultado esperado. Relacionen cada comprobación con las funcionalidades, reglas o endpoints que correspondan.

Condición inicial común: sistema levantado, realm importado, cinco espacios cargados y tokens obtenidos. Todo entra por localhost:8080.

1. Comprobación de Accesos:

Condición: Tokens de usuario1 y admin1

Acción: GET /api/espacios sin token y con usuario1; POST /api/espacios con usuario1 y con admin1

Resultado: 401, 200, 403 y 201

2. Disponibilidad:

Condición: Sin reservas mañana de 10 a 12

Acción: GET /api/reservas/disponibilidad: CLASE, 30 asistentes, PROYECTOR

Resultado: 200 con Aula 101

3. Reserva y Superposición:

Condición: Aula 101 sugerida

Acción: POST /api/reservas; repetir búsqueda; repetir reserva

Resultado: 201 CONFIRMADA; 200 vacía; 409

4. Validación de Reglas:

Condición: Espacios cargados

Acción: POST /api/reservas rompiendo cada regla; espacio 999

Resultado: 422 con el motivo; 404

5. Flujo de Estados y Roles:

Condición: Tokens de los tres usuarios

Acción: Reservar EVENTO; aprobar con admin1; aprobar de nuevo; usuario1 aprueba; usuario2 cancela; usuario1 cancela

Resultado: 201 PENDIENTE; 200 CONFIRMADA; 409; 403; 403; 200

6. Concurrencia:

Condición: Espacio libre

Acción: Dos POST /api/reservas idénticos en paralelo

Resultado: 201 y 409, sin duplicados

7. Integraciones y Tolerancia a Fallos:

Condición: Sistema funcionando

Acción: Reservar cancha con CLIMA_UMBRAL=0; con Open-Meteo caído; con ms-espacios apagado, crear y consultar /mias

Resultado: 422; 503; 503 y 200

◄ Alternativa de obtención de distancias con OSRM

Saltar a actividad

Clase 28 8 Laboratorio ►